



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

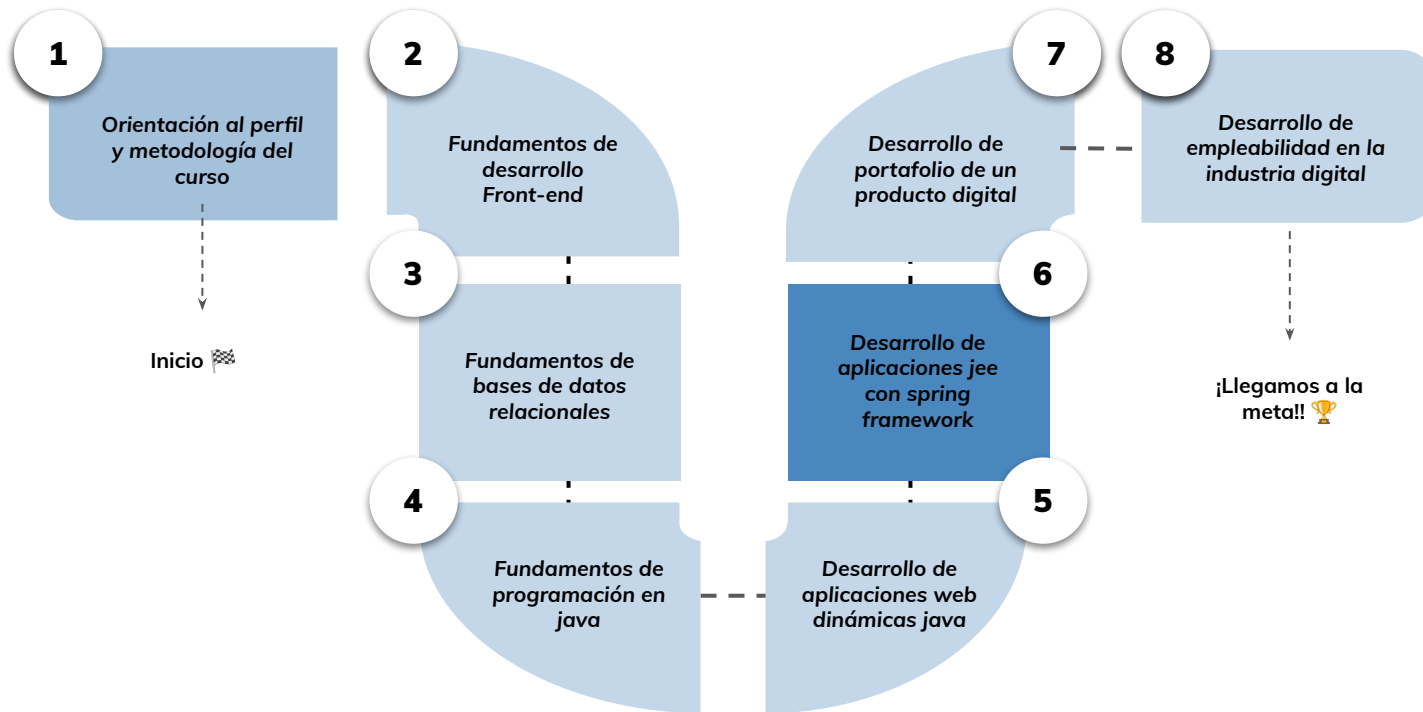


› Control de acceso mediante spring security - Parte I

Aprendizaje Esperado 4: Implementar mecanismos de seguridad utilizando spring security para controlar el acceso a los recursos del aplicativo.

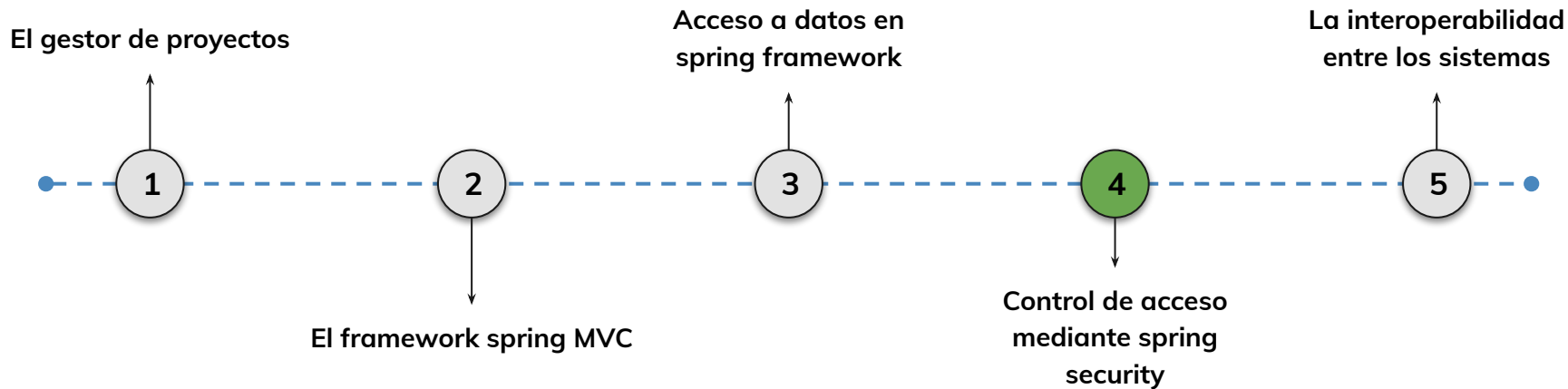
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



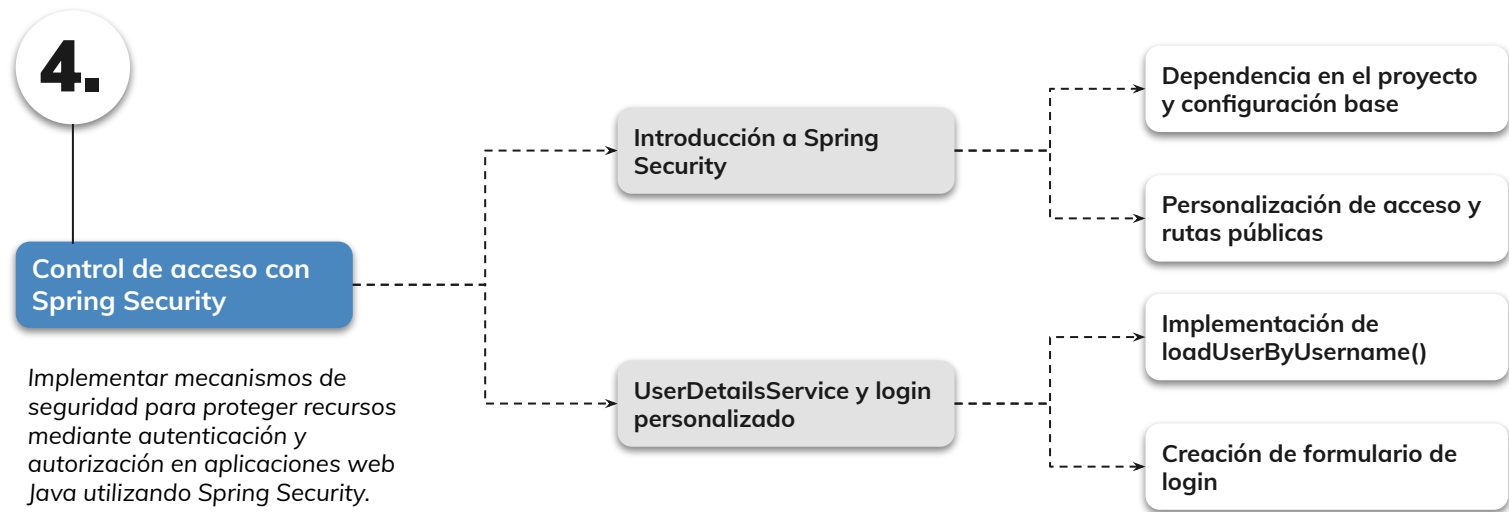
Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?



Objetivos de aprendizaje

¿Qué aprenderás?

- Comprender el funcionamiento básico de Spring Security.
- Configurar autenticación basada en formulario.
- Implementar UserDetailsService para control de usuarios.
- Proteger rutas específicas según el rol del usuario.
- Crear una configuración personalizada de login/logout.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Reemplazo de DAOs con repositorios JPA (CrudRepository) y consultas personalizadas con @Query.
- Mapeo de relaciones entre entidades con anotaciones @OneToOne y @OneToMany.
- Inyección de repositorios en servicios usando @Autowired.
- Uso de @Transactional para gestionar operaciones críticas y readOnly = true para consultas.
- Configuración de propagación y aislamiento transaccional en servicios.

Control de acceso mediante spring security

< > Incorporando Spring Security al proyecto

En este ejemplo, configuramos reglas de autorización y autenticación.

Permitimos el acceso a todas las URL que coincidan con `"/public/**"`, requerimos autenticación para cualquier otra URL, configuramos una página de inicio de sesión personalizada, y permitimos que todos los usuarios cierren sesión.

```
@Configuration
@EnableWebSecurity
public class SeguridadWeb extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests()
            .antMatchers("/public/**").permitAll()
            .anyRequest().authenticated()
            .and()
            .formLogin(login -> login
                .loginPage("/login")
                .permitAll())
            .logout(logout -> logout
                .permitAll());
    }
}
```



LIVE CODING

Seguridad en la Wallet

- Vamos a agregar la dependencia de Spring Security en el proyecto AlkeWallet
- Además, agregaremos las configuraciones necesarias en el archivo **application.properties**
- También crearemos una nueva clase: **SecurityConfig**, que debe extender de `WebSecurityConfigurerAdapter` y debe sobrescribir el método `configure` para personalizarlo.

Tiempo: 25 minutos

➤ Administrando los usuarios del programa

< > Administrando los usuarios del programa

En Spring Security, la interfaz `UserDetailsService` juega un papel fundamental en la **autenticación**. Su función principal es cargar los detalles del usuario a partir de algún origen de datos, como una base de datos, un servicio web o incluso una configuración en memoria.

Al implementar esta interfaz, puedes personalizar la carga de usuarios y roles según las necesidades de tu aplicación.



< > Administrando los usuarios del programa

Para que Spring pueda obtener la información de un usuario durante el proceso de **login**, se solicitará al conjunto de clases Java registradas en el contexto de Spring, que alguna implemente la interfaz `UserDetailsService`.

Esta interfaz tiene un único método de obligada implementación que es aquel que devuelve los detalles de un usuario `UserDetails` dado un **nombre de usuario**, el que trata de ingresar en la aplicación. Por tanto será obligatorio implementar un **bean** con esa interfaz dando cuerpo a ese método.



< > Administrando los usuarios del programa

Características clave de `UserDetailsService`:

1. **Carga de Detalles del Usuario** : la interfaz `UserDetailsService` contiene un solo método, `loadUserByUsername`, que se encarga de cargar los detalles del usuario según su nombre de usuario.
2. **Retorno de UserDetails** : el método `loadUserByUsername` devuelve un objeto `UserDetails`, que representa los detalles del usuario, incluyendo su nombre de usuario, contraseña encriptada y roles asignados.
3. **Integración con Spring Security** : la implementación de `UserDetailsService` se integra en la configuración de Spring Security para proporcionar detalles de usuario durante el proceso de autenticación.



< > Administrando los usuarios del programa

Veamos un ejemplo práctico dónde la interfaz `UserDetailsService` será implementada por una clase que maneja el servicio de Usuario: `UserService`, y el método a sobrescribir se llama `loadUserByUsername()`.

```
@Service
public class UserService implements UserDetailsService {

    @Autowired
    private UserRepository userRepository;
```



< > Administrando los usuarios del programa

También debemos tener en cuenta que la interfaz de repositorio (en este ejemplo: `UserRepository`) extiende de `JpaRepository` o `CrudRepository`:

```
@Repository
public interface UserRepository extends JpaRepository<UserEntity, String> {

    @Query("SELECT u FROM UserEntity u WHERE u.email = :email")
    public UserEntity searchByEmail(@Param("email")String email);
}
```

En este repositorio hemos declarado un pequeño método personalizado de **búsqueda de usuario por email** (`searchByEmail(String email)`).

< > Administrando los usuarios del programa

Finalmente, podemos ver como quedaría el método `loadUserByUsername` en la clase de servicio. Avancemos para poder ver la explicación de cada línea de código.

```
149 • @Override
150 public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
151
152     UserEntity user = userRepository.searchByEmail(email);
153
154     if (user != null) {
155
156         List<GrantedAuthority> permissions = new ArrayList<>();
157
158         GrantedAuthority p = new SimpleGrantedAuthority("ROLE_" + user.getRole().toString());
159
160         permissions.add(p);
161
162         ServletRequestAttributes attr = (ServletRequestAttributes) RequestContextHolder.currentRequestAttributes();
163
164         HttpSession session = attr.getRequest().getSession(true);
165
166         session.setAttribute("usuariosession", user);
167
168         return new User(user.getEmail(), user.getPassword(), permissions);
169     } else {
170
171         return null;
172     }
173 }
174 }
175 }
```

< > Administrando los usuarios del programa

Vamos a referenciar las líneas del código para explicar el método :

- **152** - Instanciamos un objeto del tipo **UserEntity**, haciendo uso del método **searchByEmail(email)** de la clase **UserRepository**.
- **154** - Verificamos que el objeto Usuario **no esté nulo** .

```
149 • @Override
150 public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
151
152     UserEntity user = userRepository.searchByEmail(email);
153
154     if (user != null) {
155
156         List<GrantedAuthority> permissions = new ArrayList<>();
157
158         GrantedAuthority p = new SimpleGrantedAuthority("ROLE_" + user.getRole().toString());
159         permissions.add(p);
160
161
162         ServletRequestAttributes attr = (ServletRequestAttributes) RequestContextHolder.currentRequestAttributes();
163
164         HttpSession session = attr.getRequest().getSession(true);
165
166         session.setAttribute("usuariosession", user);
167
168         return new User(user.getEmail(), user.getPassword(), permissions);
169
170     } else {
171
172         return null;
173     }
174 }
175
```



< > Administrando los usuarios del programa

- **156** - Otorgamos permisos según el rol que tenga cada usuario. Si el usuario existe, creamos una lista de permisos llamada **permissions**.
- **158** - Creamos un objeto **GrantedAuthority 'p'** y concatenamos la palabra **ROLE_** + el rol del usuario.

```
149 @Override
150 public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
151
152     UserEntity user = userRepository.searchByEmail(email);
153
154     if (user != null) {
155
156         List<GrantedAuthority> permissions = new ArrayList<>();
157
158         GrantedAuthority p = new SimpleGrantedAuthority("ROLE_" + user.getRole().toString());
159
160         permissions.add(p);
161
162         ServletRequestAttributes attr = (ServletRequestAttributes) RequestContextHolder.currentRequestAttributes();
163
164         HttpSession session = attr.getRequest().getSession(true);
165
166         session.setAttribute("usuariosession", user);
167
168         return new User(user.getEmail(), user.getPassword(), permissions);
169
170     } else {
171
172         return null;
173     }
174 }
175
```



< > Administrando los usuarios del programa

- **160** - Agregamos el objeto 'p' a la lista de permisos.
- **162/164** - Utilizamos los atributos que nos otorga el pedido al **servlet**, para poder guardar la información de nuestra **HttpSession**.

```
149 @Override
150 public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
151
152     UserEntity user = userRepository.searchByEmail(email);
153
154     if (user != null) {
155
156         List<GrantedAuthority> permissions = new ArrayList<>();
157
158         GrantedAuthority p = new SimpleGrantedAuthority("ROLE_" + user.getRole().toString());
159
160         permissions.add(p);
161
162         ServletRequestAttributes attr = (ServletRequestAttributes) RequestContextHolder.currentRequestAttributes();
163
164         HttpSession session = attr.getRequest().getSession(true);
165
166         session.setAttribute("usuariosession", user);
167
168         return new User(user.getEmail(), user.getPassword(), permissions);
169
170     } else {
171
172         return null;
173     }
174 }
175
```



< > Administrando los usuarios del programa

- **168** - por último, retornamos un User con su email, contraseña y permisos!

```
149 @Override
150 public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
151
152     UserEntity user = userRepository.searchByEmail(email);
153
154     if (user != null) {
155
156         List<GrantedAuthority> permissions = new ArrayList<>();
157
158         GrantedAuthority p = new SimpleGrantedAuthority("ROLE_" + user.getRole().toString());
159
160         permissions.add(p);
161
162         ServletRequestAttributes attr = (ServletRequestAttributes) RequestContextHolder.currentRequestAttributes();
163
164         HttpSession session = attr.getRequest().getSession(true);
165
166         session.setAttribute("usuariosession", user);
167
168         return new User(user.getEmail(), user.getPassword(), permissions);
169
170     } else {
171
172         return null;
173     }
174 }
175
```



< > Administrando los usuarios del programa

La implementación de este bean **dependerá de dónde tengamos registrados los usuarios**.

Normalmente éstos estarán en una tabla de la base de datos, así que ya sea con SQL nativo o usando JPA con EntityManager deberemos consultar la fila asociada según el nombre de usuario.

Una vez localizado sólo restará convertir o mapear ese objeto a uno esperado por Spring, del tipo **UserDetails**.



➤ Creando un formulario de login

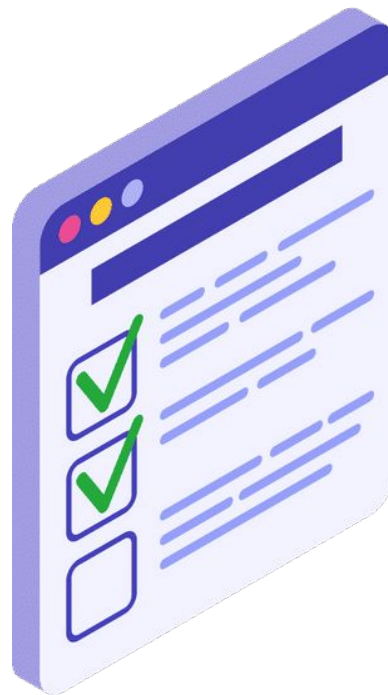


Creando un formulario de login

Spring Security facilita la implementación de un formulario de **login** personalizado.

Al configurar Spring Security, puedes definir la página de login, las URL de autenticación, y personalizar el comportamiento del formulario.

La autenticación basada en formularios es una de las estrategias más comunes, y Spring Security la hace fácil de implementar.





Creando un formulario de login

Dentro de la clase que extiende de `WebSecurityConfigurerAdapter` (en nuestro ejemplo, la clase `SecurityConfig`), vamos a sobrescribir el método `configure()`:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                .antMatchers("/public/**").permitAll()
                .anyRequest().authenticated()
            .and()
            .formLogin()
                .loginPage("/login") // Ruta a la página de login personalizada
                .permitAll() // Permite a todos acceder a la página de login
            .and()
            .logout()
                .permitAll();
    }
}
```



Creando un formulario de login

`.anyRequest().authenticated()` hace referencia a que se requiere estar autenticado para todas las peticiones.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                .antMatchers("/public/**").permitAll()
                .anyRequest().authenticated()
            .and()
            .formLogin()
                .loginPage("/login") // Ruta a la página de login personalizada
                .permitAll() // Permite a todos acceder a la página de login
            .and()
            .logout()
                .permitAll();
    }
}
```



Creando un formulario de login

`.formLogin().loginPage("/login").permitAll()` refiere a que el formulario será definido en la página `login.jsp` (o `login.html`) que crearemos más adelante, las peticiones a esta página no requieren autenticación, cualquier usuario puede acceder a ella.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                .antMatchers("/public/**").permitAll()
                .anyRequest().authenticated()
                .and()
            .formLogin()
                .loginPage("/login") // Ruta a la página de login personalizada
                .permitAll() // Permite a todos acceder a la página de login
                .and()
            .logout()
                .permitAll();
    }
}
```



Creando un formulario de login

`.logout().permitAll()` activa el cierre de sesión mediante la URL `/logout`. Cualquier usuario puede acceder a esta URL.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                .antMatchers("/public/**").permitAll()
                .anyRequest().authenticated()
            .and()
            .formLogin()
                .loginPage("/login") // Ruta a la página de login personalizada
                .permitAll() // Permite a todos acceder a la página de login
            .and()
            .logout()
                .permitAll();
    }
}
```



Creando un formulario de login

Lo que prosigue es crear el formulario JSP (o HTML), para que funcione correctamente debe tener dos campos: username y password, estos deben ser enviados usando el método POST a la URL **"/login"** (como se especificó en la configuración).

```
<!-- login.jsp -->
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Login</title>
</head>
<body>
  <h2>Login</h2>
  <form action="<c:url value='/login' />" method="post">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username" required>
    <br>
    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required>
    <br>
    <button type="submit">Login</button>
  </form>
</body>
</html>
```





Creando un formulario de login

Finalmente, crea un controlador simple para manejar la solicitud de la página de login. Este controlador simplemente devuelve el nombre de la vista (el nombre del archivo JSP sin extensión) cuando se accede a la URL /login.

```
@Controller
public class LoginController {

    @GetMapping("/login")
    public String login() {
        return "login";
    }
}
```

Ahora, puedes ejecutar tu aplicación y acceder a la página de login personalizada en <http://localhost:8080/login>. Ingresa las credenciales de usuario configuradas en la clase SecurityConfig y observa cómo Spring Security maneja la autenticación.

Cerrando sesión con Spring Security



Spring Security permite implementar fácilmente el cierre de sesión mediante la configuración del método `logout()` dentro de la clase que extiende `WebSecurityConfigurerAdapter`.

En la clase `SecurityConfig`, el método `configure()` incluye:

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http.authorizeRequests()
        .anyRequest().authenticated()
        .and()
        .formLogin().loginPage("/login").permitAll()
        .and()
        .logout().permitAll();
}
```

La línea `.logout().permitAll()` habilita el cierre de sesión para todos los usuarios, permitiendo acceder a la URL `/logout` sin autenticación previa.

Cerrando sesión con Spring Security



Una vez ejecutada la acción de logout, el usuario es redirigido automáticamente a la página `/login?logout`, donde puede mostrarse un mensaje confirmando que la sesión se ha cerrado correctamente:

```
<c:if test="${param.logout != null}">
  <p class="logout">La sesión ha sido cerrada correctamente.</p>
</c:if>
```

De esta forma, se completa el ciclo de autenticación y cierre de sesión de manera controlada, manteniendo la seguridad y una experiencia de usuario clara y consistente.

Live Coding

¿En qué consistirá la Demo?

Vamos a crear un login básico donde un usuario con rol `USER` puede acceder a su perfil, pero no a zonas administrativas.

Este ejercicio nos permitirá simular el comportamiento de un login completo con rutas protegidas por perfil.

1. Crear un usuario en base de datos o memoria con rol `USER`.
2. Implementar `UserDetailsService` y asignar `ROLE_USER`.
3. Configurar en `SecurityConfig`:
 - a. `.antMatchers("/mi-perfil").hasRole("USER")`
 - b. `.antMatchers("/login", "/inicio").permitAll()`
4. Crear formulario simple de login (con campos `username` y `password`).
5. Probar que el usuario solo puede acceder a `/mi-perfil`.
6. Verificar que recibe acceso denegado (403) al intentar ingresar a `/admin/**`.
7. Cerrar sesión con `/logout`.
8. Mostrar flujo completo de login/logout.

Live Coding

¿En qué consistirá la Demo?

En esta demo vamos a trabajar con un usuario con rol MANAGER, que tiene más permisos que un USER, pero no accede a `/admin/**`.

1. Crear un usuario en base de datos o memoria con rol MANAGER
2. Implementar UserDetailsService y asignar ROLE_MANAGER.
3. Configurar en SecurityConfig:
 - a. `.antMatchers("/panel-gestor").hasRole("MANAGER")`
 - b. `.antMatchers("/reportes").hasRole("MANAGER")`
 - c. `.antMatchers("/admin/**").denyAll()`
4. Crear una vista simple para `/panel-gestor`.
5. Verificar que recibe acceso denegado (403) al intentar ingresar a `/admin/**`.
6. Cerrar sesión con `/logout`.

Live Coding

¿En qué consistirá la Demo?

Aquí implementaremos acceso completo para un usuario con rol ADMIN, que puede gestionar todo el sistema.

1. Crear usuario con rol ADMIN
2. Implementar UserDetailsService para devolver ROLE_ADMIN.
3. Configurar en SecurityConfig:
 - a. `.antMatchers("/admin/**").hasRole("ADMIN")`
 - b. `.antMatchers("/panel-gestor").hasAnyRole("ADMIN", "MANAGER")`
 - c. `.antMatchers("/mi-perfil").authenticated()`
4. Probar acceso exitoso del admin a:
 - a. `/admin/usuarios`
 - b. `/admin/config`
 - c. `/panel-gestor`
 - d. `/mi-perfil`

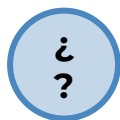
LIVE CODING

Login en la Wallet

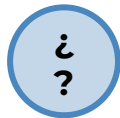
- Vamos a agregar las configuraciones de autenticación para el proyecto AlkeWallet.
 1. Implementar la interfaz **UserDetailsService** en el servicio de usuario y sobrescribir el método **loadUserByUsername**.
 2. Personalizar la URL de **login** y **logout** en la clase de configuración.

Tiempo: 20 minutos

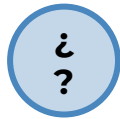
#Momentode Preguntas...



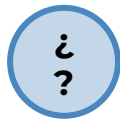
¿Qué ventajas ofrece Spring Security frente a una implementación casera de seguridad?



¿Cuál es el propósito del método `loadUserByUsername()`?



¿Qué sucede si no se define correctamente el formulario de login?



¿Qué tipo de objeto debe retornar `UserDetailsService`?



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

Seguridad de acceso en Biblioteca Digital

Seguridad de acceso en Biblioteca Digital

Contexto: 🙌

Estás desarrollando una plataforma web para una biblioteca digital. En ella, los usuarios pueden consultar libros y acceder a material exclusivo según su rol (lector, bibliotecario o administrador). El equipo necesita que protejas los recursos sensibles del sistema permitiendo el ingreso solo a usuarios autenticados, mediante un formulario de inicio de sesión.

Consigna: 📝

Implementa mecanismos de seguridad en la plataforma de biblioteca utilizando Spring Security. Configura reglas de autenticación y autorización, personaliza la URL de acceso y crea un formulario para el login. Además, asegúrate de que los recursos públicos sigan siendo accesibles sin autenticación.

Tiempo 🕒: 30 minutos

Seguridad de acceso en Biblioteca Digital

Paso a paso:

1. Agrega la dependencia de Spring Security al archivo pom.xml.
2. Crea la clase SecurityConfig y sobrescribe el método configure().
3. Define rutas públicas (por ejemplo, /libros/publicos) y privadas (por ejemplo, /admin/**).
4. Implementa UserDetailsService en el servicio UsuarioService.
5. Crea un formulario con los campos username y password.
6. Configura correctamente las URLs de login y logout.
7. Ejecuta la aplicación y accede al login personalizado en /login.

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Incorporar Spring Security en un proyecto Java
- ✓ Crear configuraciones de seguridad personalizadas
- ✓ Implementar la autenticación basada en usuarios
- ✓ Integrar un formulario de login personalizado
- ✓ Comprender la interfaz UserDetailsService
- ✓ Aplicar permisos según roles de usuario



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

